

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Containerization with Docker

Date 2026/5/21 Class AI 1

Name Weijie Xu ID 202383910029

一、 Experimental purpose

(1) To Understand the Basic Concept of Docker Containerization

The first purpose of this experiment is to understand what Docker containerization means and how it is used in cloud computing. Through this lab, students learn that a container is a lightweight and isolated runtime environment that can package an application together with the files and configurations it needs. Compared with running a web application directly on the host machine, Docker provides a more standardized way to deploy and manage applications. In this experiment, the web page is not only stored as a normal index.html file, but is also packaged into a Docker image and executed inside a container. This helps students build a basic understanding of how modern cloud applications are packaged, delivered, and executed in a controlled environment.

(2) To Learn How to Build a Docker Image for a Web Application

Another important purpose of this experiment is to learn how to build a Docker image from a simple web application. By creating a Dockerfile, students can understand how Docker uses a set of instructions to define the application environment. In this lab, the Nginx image is used as the base image, and the local index.html file is copied into the default web directory of Nginx. Through the docker build command, the source file and server environment are packaged into a reusable image. This process helps students understand that a Docker image is not just a file, but a complete application snapshot that can be used to create one or more running containers.

(3) To Practice Running and Accessing a Containerized Web Server

This experiment also aims to help students practice running a containerized web server. After the image is built, students use the docker run command to start a

container based on the image. The container runs an Nginx web server internally and serves the personal index.html page. By opening the browser through the forwarded port in Codespaces, students can verify that the website is successfully served from inside the Docker container. This process connects the theoretical concept of container deployment with a visible running result, making it easier to understand how cloud services are launched and accessed.

(4) To Understand Port Mapping Between the Host and the Container

A key purpose of the experiment is to understand how network port mapping works between the host machine and the container. In this lab, the container listens on port 80, while the host exposes the service through port 8080. The command `docker run -d -p 8080:80` shows that requests sent to the host's port 8080 are forwarded to port 80 inside the container. This helps students understand that containers have their own internal network environment, and external users cannot directly access container services unless proper port mapping is configured. This knowledge is important for deploying cloud applications, because real cloud services usually depend on correct network configuration.

(5) To Explore Immutable Infrastructure in Cloud Computing

This experiment is designed to help students understand the concept of immutable infrastructure. After the container is started, students modify the original index.html file and refresh the running website. The website does not change automatically because the running container is based on the previously built image. This shows that a Docker image is a fixed snapshot of the application at the time it was built. To apply new changes, the image must be rebuilt and the container must be restarted. This behavior is useful in cloud computing because it improves consistency, reduces unexpected changes, and supports the principle of “build once and run anywhere”.

(6) To Observe Container Isolation, Resource Usage, and Scalability

The final purpose of this experiment is to observe several important cloud computing features of containers, including isolation, efficiency, and scalability. By entering the running container with `docker exec`, students can see that the container has its own file system, separate from the Codespaces host environment. By using `docker stats`, students can observe CPU and memory usage and understand why containers are more lightweight than traditional virtual machines. By running a second container from the same image on another port, students can also experience basic service scaling. At the end of the lab, removing unused containers further helps

students understand the importance of resource cleanup in a pay-as-you-go cloud environment.

二、Experiment Contents

- In this experiment, I first opened my GitHub repository and entered the Codespaces environment to prepare a cloud-based development workspace. This step provided a convenient online environment for writing files, running terminal commands, and testing Docker services. After the Codespace was created successfully, I checked whether the Docker environment was available by running the `docker version` command. The purpose of this operation was to confirm that both the Docker Client and Docker Server were running properly before building and launching containers.
- After verifying the Docker environment, I created a new file named `Dockerfile` in the project directory. This file was used to describe how the Docker image should be built. In the `Dockerfile`, I selected `nginx:alpine` as the base image because Nginx can serve static web pages efficiently, and the Alpine version is lightweight. Then, I copied the local `index.html` file into the default Nginx web directory inside the container and exposed port 80 to indicate that the container would provide web service through this port.
- Next, I built the Docker image by using the command `docker build -t cloud-lab-image:v1 ..`. This command packaged the web page and the Nginx runtime environment into a reusable Docker image. The tag `cloud-lab-image:v1` was used to give the image a clear name and version. During this process, Docker read the instructions in the `Dockerfile`, downloaded the required base image if necessary, copied the web page into the correct directory, and generated a new image that could be used to create containers.
- After the image was built, I started a running container with the command `docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1`. This command launched the container in detached mode and mapped port 8080 of the host environment to port 80 inside the container. Then I opened the forwarded port in the browser through Codespaces. If the web page was displayed correctly, it proved that the containerized Nginx server was running successfully and that the local `index.html` file had been served from inside the Docker container.
- To test the immutability of Docker containers, I modified the original `index.html` file, such as changing the page text or background color, and then refreshed the

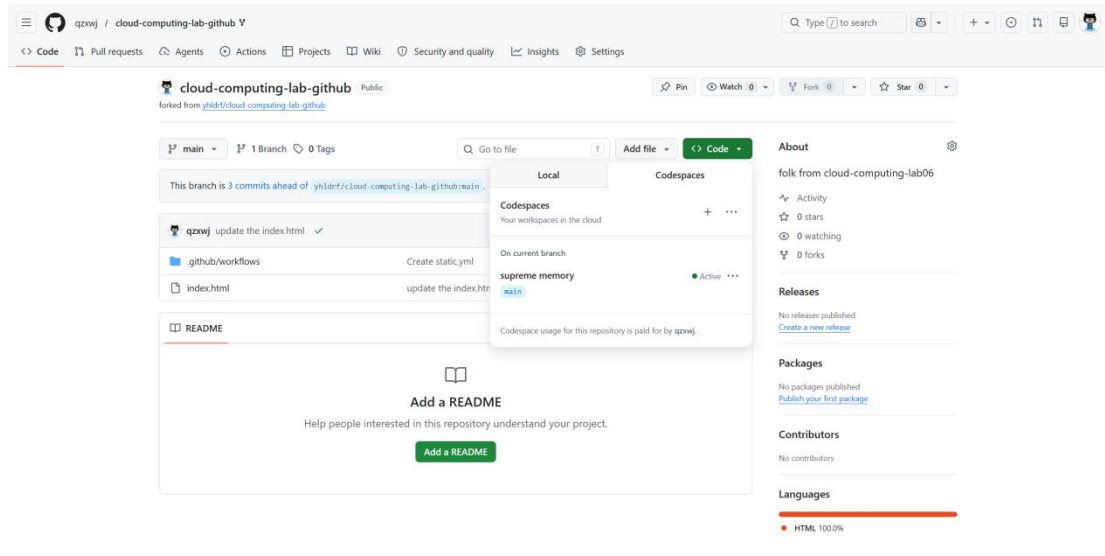
browser page served by the running container. The page did not update automatically because the container was created from the previously built image. This showed that the running container used a fixed snapshot of the application at build time. To make the modification effective, the image needed to be rebuilt and the container needed to be restarted, which reflected the idea of immutable infrastructure in cloud computing.

- I also explored the internal file system of the running container by using the command `docker exec -it my-web-container sh`. After entering the container shell, I listed the files in `/usr/share/nginx/html/` and used the `cat` command to view the copied `index.html` file. This step helped me understand that a container has its own isolated file system, which is separated from the host Codespaces environment. Then, I used `docker stats` to monitor container resource usage and observed CPU and memory consumption, which showed that containers are lightweight and efficient compared with traditional virtual machines.
- Finally, I practiced scaling and resource cleanup. I created a second container from the same image by mapping another host port, such as 8081, to the container's port 80. By opening both ports, I could observe that two independent containers were serving the same web page at the same time. Then, I intentionally tried to reuse an occupied port and observed the port allocation error, which helped me understand the importance of port management. At the end of the experiment, I stopped and removed the containers, checked that no running containers remained, and used Git commands to save and push the lab changes to GitHub.

三、 Detailed steps of the experiment

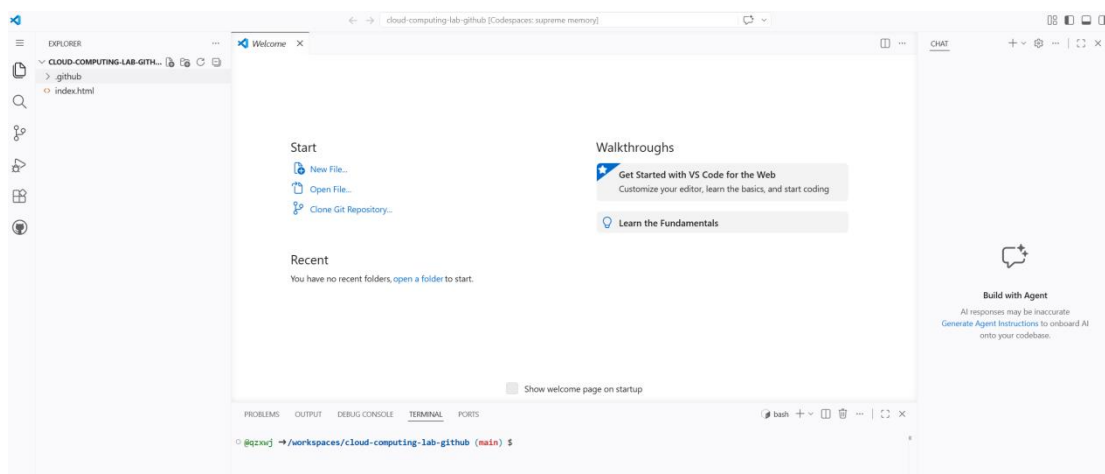
Environment Setup 1: Open GitHub Repository

This screenshot shows the beginning of the lab environment setup. I opened my GitHub repository in the browser and confirmed that the project files were already stored in the remote repository. This repository is the foundation for the whole experiment because it provides the source files, version history, and online workspace entry point. Before using Docker, I first needed to make sure that I was working in the correct project rather than an unrelated local folder. From a cloud computing perspective, GitHub acts as a remote resource management platform. The code can be accessed through the Internet, connected to Codespaces, modified online, and later pushed back for permanent storage, which reflects the separation between local devices and cloud-hosted development resources.



Environment Setup 2: Open Codespaces from Code Menu

In this step, I clicked the Code button in the GitHub repository and selected Codespaces. This operation is more than simply opening an editor; it links the repository with a cloud-based development environment. Codespaces provides a remote Linux workspace, terminal, file explorer, source editor, Docker support, and port forwarding service in the browser. This means the experimental environment can be prepared without manually installing Docker, Nginx, or a Linux virtual machine on my laptop. The screenshot records the transition from code hosting to cloud development. The key cloud concept is on-demand development environment provisioning, where computing resources are created when needed and attached directly to the repository.



Environment Setup 3: Create Codespace

After opening the Codespaces option, I clicked Create codespace to start a new cloud development workspace. GitHub then allocated and initialized a remote environment connected with my repository. This environment behaves like a small cloud virtual machine for development: it has a terminal, file system, editor, network forwarding, and the tools required to run commands. This step is important because all later Docker operations are executed inside this cloud workspace rather than on my local operating system. It demonstrates how cloud platforms reduce environment differences between users. Instead of each student configuring a machine separately, Codespaces provides a consistent development environment based on the repository.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker version
Client:
  Version:           29.3.0-1
  API version:       1.54
  Go version:        go1.26.0
  Git commit:        5927d80c76b3ce5cf782be818922966e8a0d87a3
  Built:             Fri Mar  6 18:50:47 2026
  OS/Arch:           linux/amd64
  Context:           default

Server:
  Engine:
    Version:          29.3.0-1
    API version:       1.54 (minimum version 1.40)
    Go version:        go1.26.0
    Git commit:        83bca512aa7ffc1bb4f37ce1107e0d3e3489ad43
    Built:             Fri Mar  6 18:29:13 2026
    OS/Arch:           linux/amd64
    Experimental:      false
  containerd:
    Version:          2.2.1-1
    GitCommit:        dea7da592f5d1d2b7755e3a161be07f43fad8f75
  runc:
    Version:          1.4.0-2
    GitCommit:        8bd78a9977e604c4d5f67a7415d7b8b8c109cdc4
  docker-init:
    Version:          0.19.0
    GitCommit:
```

Step 1: Verify Docker Environment

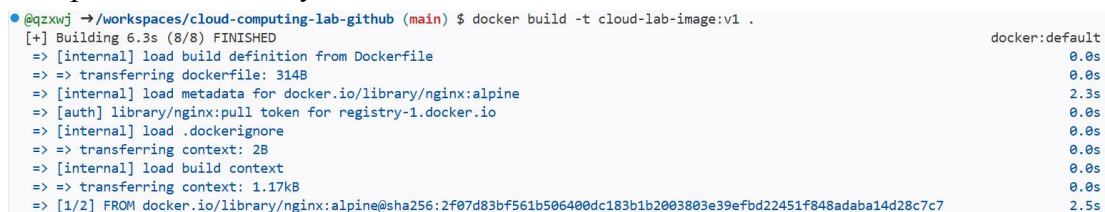
In the Codespaces terminal, I ran the command `docker version` to check whether Docker was available and whether the Docker daemon was running correctly. The expected result includes both Client and Server version information. The Client section represents the command-line interface that receives my Docker commands, while the Server section proves that the Docker engine is active in the background and ready to build images or run containers. This check is necessary because later steps such as `docker build`, `docker run`, `docker exec`, and `docker stats` all depend on a working Docker service. In cloud computing terms, this step verifies that the cloud workspace already contains the container runtime needed for cloud-native application deployment.

A screenshot of a code editor window showing a Dockerfile. The editor has a tab labeled 'Dockerfile U' and a sidebar with a 'Dockerfile' icon. The Dockerfile content is as follows:

```
1 # 使用 Nginx (一种常用的 Web 服务器) 作为基础镜像
2 FROM nginx:alpine
3
4 # 将本地的 index.html 复制到容器内部的 Nginx 网页目录中
5 COPY index.html /usr/share/nginx/html/index.html
6
7 # 告诉 Docker, 该容器在运行时监听 80 端口
8 EXPOSE 80
```

Step 2: Create Dockerfile

This screenshot records the creation of a new file named Dockerfile in the Codespaces file explorer. The name must be exactly Dockerfile, with the correct capitalization and no file extension, because Docker searches for this default file name when building an image from the current directory. Creating this file is the first step in turning a normal web project into a containerized application. The Dockerfile works like a build recipe that explains what base environment should be used, which files should be copied into the image, and which port the application will use. This reflects the cloud-native idea of infrastructure as code, where deployment instructions are written into files and can be repeated consistently.

A screenshot of a terminal window showing the output of a Docker build command. The command is `docker build -t cloud-lab-image:v1 .`. The output shows the build process, including loading the build definition, transferring the Dockerfile, loading metadata for the nginx:alpine image, and finally building the image. The build is successful, and the image is tagged as cloud-lab-image:v1.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-lab-image:v1 .
[+] Building 6.3s (8/8) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 314B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine  2.3s
=> [auth] library/nginx:pull token for registry-1.docker.io    0.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                   0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 1.17kB                               0.0s
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:2f07d83bf561b506400dc183b1b2003803e39efbd22451f848adaba14d28c7c7 2.5s
```

Step 2: Add Dockerfile Instructions

After creating the Dockerfile, I pasted the required instructions into it. The file uses nginx:alpine as the base image, which provides a lightweight Nginx web server environment. It then copies the local index.html file into /usr/share/nginx/html/index.html, which is the default web directory used by Nginx inside the container. The EXPOSE 80 instruction documents that the containerized service listens on port 80 at runtime. These instructions combine my webpage with a ready-made web server runtime. This step shows why Docker images are portable: the application content, runtime dependency, and basic network expectation are described together, so the same image can run in different cloud environments with consistent behavior.

A screenshot of a terminal window showing the output of a Docker run command. The command is `docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1`. The output shows the container ID and the command being executed.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1
43af25102ce79d09dc56fd9821793e73d8f53b792009d591e8c2620424295963
```

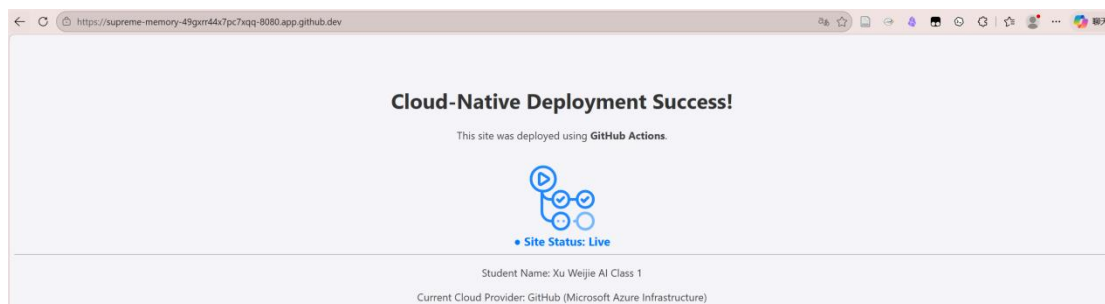
Step 3: Build the Docker Image

In this step, I ran `docker build -t cloud-lab-image:v1 .` to build a Docker image from the Dockerfile. The `-t` option assigns a meaningful image name and version tag, making it easier to run and manage later. The dot at the end is also important because it tells Docker to use the current folder as the build context, including the Dockerfile and `index.html` file. During the build process, Docker reads each instruction, prepares the base Nginx layer, copies my web page into the image, and produces a reusable image snapshot. This step converts source code and configuration into a deployable artifact, which is a common workflow in cloud-native deployment pipelines.

Port	Forwarded Address	Running Process	Visibility	Origin
8080	https://supr...	/usr/libexec/docker/docker-proxy -proto tcp -ho...	Private	Auto Forwarded

Step 3: Confirm Image Build Result

This screenshot confirms that the Docker image build process completed successfully. A successful build means Docker has produced an image named `cloud-lab-image:v1` that can be used as a template for running containers. At this stage, the image itself is not yet a running website; it is a packaged snapshot that contains the web server and the copied `index.html` file. This distinction is important in container technology. An image is similar to a blueprint or template, while a container is an actual running instance created from that image. In cloud computing, preparing a reliable image before deployment helps ensure that every later instance starts from the same known state.



Step 4: Run the Container

After building the image, I ran `docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1` to start a running container. The `-d` option makes the container run in the background, so the terminal remains available for other commands. The `--name` option assigns a clear name to the container, which makes later commands such as

docker exec, docker stop, and docker rm easier to use. The port mapping -p 8080:80 connects port 8080 of the Codespaces host to port 80 inside the container. This step turns the static image into an active web service. It also demonstrates the relationship between a cloud host, container network, and exposed application port.

```
<body>
  <h1>Cloud-Native Deployment Updated!</h1>

  <p>This site was deployed using <b>GitHub Actions</b>.</p>

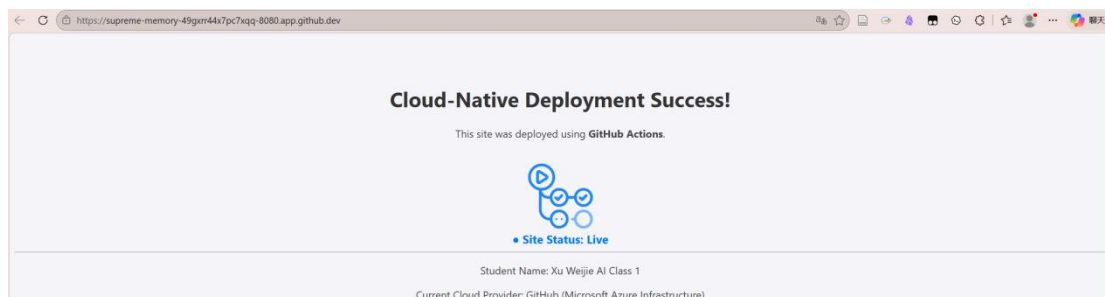
  <div class="status">• Site Status: Live</div>

  <hr>

  <p>Student Name: Xu Weijie AI Class 1</p>
  <p>Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)</p>
</body>
```

Step 5: Access the Web Server

After the container started, Codespaces detected that a service was running on port 8080 and provided an option to open it in the browser. I opened the forwarded address and verified that my personal index.html page was displayed successfully. This proves that the Nginx server inside the container is responding to HTTP requests and serving the copied web page. The important observation is that the browser is not reading index.html directly from the Codespaces file explorer. Instead, the request enters through the forwarded host port, reaches the container's port 80, and is handled by Nginx. This step demonstrates how cloud platforms expose containerized services through managed port forwarding.



Step 6: Modify Source Code for Immutability Test

To test container immutability, I modified the source index.html file in the Codespaces editor, such as changing the page text or background color. This update changes the current source file in the repository workspace, but it does not automatically change the Docker image that was already built earlier. The purpose of making this modification is to create a visible difference between the editable source

code and the file snapshot stored inside the running container. This step prepares the key comparison in the experiment. It helps show that a containerized application does not continuously read from the original project folder unless a volume mount or rebuild workflow is explicitly configured.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker exec -it my-web-container sh
/ # ls /usr/share/nginx/html/
50x.html    index.html
```

Step 6: Verify the Running Container Does Not Change

After changing the source file, I refreshed the browser page served from port 8080. The website did not update, which proves that the running container is still using the older index.html copied into the Docker image during the build step. This is the central observation for container immutability. A Docker image is a fixed snapshot of an application and its runtime environment, and a container created from that image keeps using that snapshot. Therefore, editing the source file after the container has started does not affect the running service directly. To publish the new content, I would need to rebuild the image and restart the container from the updated image.

```
/ # cat /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Cloud Computing Lab</title>
  <style>
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      text-align: center;
      padding-top: 50px;
      background-color: #f4f4f9;
    }

    .status {
      color: #007bff;
      font-weight: bold;
      font-size: 18px;
    }
  </style>
</head>
<body>
  <div class="status">
    Cloud Computing Lab
  </div>
</body>
</html>
```

Step 7: Enter the Container Shell

In this step, I used `docker exec -it my-web-container sh` to open an interactive shell inside the running container. The `docker exec` command runs a new command inside an existing container, while `-it` keeps the session interactive so I can type commands and view results. Once inside, the prompt changes, showing that I am no longer operating only in the Codespaces host terminal. This step is useful because it allows direct inspection of the container's internal environment. It demonstrates that a container is like a lightweight isolated computer with its own process space and file system, even though it shares the host machine's kernel.

```
/ # exit
@qzxwj → /workspaces/cloud-computing-lab-github (main) $
```

Step 7: Explore Container Files

After entering the container shell, I listed the files in `/usr/share/nginx/html/` and used `cat` to view the `index.html` file stored inside the container. This confirms that the webpage served by Nginx comes from the container's internal web directory. Comparing this file with the edited source file in Codespaces helps explain why the browser page did not update automatically. The container has its own file system that was created from the Docker image, so it remains separate from the current project folder. This step demonstrates isolation, one of the most important container concepts, because application files, runtime paths, and processes are organized inside the container boundary.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
43af25102ce7	my-web-container	0.00%	3.441MiB / 7.758GiB	0.04%	30.4kB / 9.58kB	0B / 24.6kB	3

Step 8: Monitor Container Resource Usage

I ran `docker stats` to observe the real-time resource usage of the running container. The output table includes CPU percentage, memory usage, network I/O, block I/O, and container name. This monitoring step connects the experiment with the economic side of cloud computing because cloud providers often charge based on resource consumption. A simple Nginx container usually uses very little memory compared with a traditional virtual machine, because containers share the host operating system kernel instead of running a full guest OS. This shows why containers are efficient for cloud deployment. Many lightweight service instances can run on the same host while consuming fewer resources.

```
^C@qzxnj → /workspaces/cloud-computing-lab-github (main) docker run -d -p 8081:80 --name second-container cloud-lab-image:v1v1
4ad7f5bcec1f7cdb44cc7236f03b45b6148062f7b0cdec1d12ced46b94276989
```

Step 9: Run a Second Container Instance

To test scaling, I launched a second container from the same image and mapped it to a different host port, such as `8081:80`. This created another running instance of the same web service without rebuilding the image. The command demonstrates how quickly containers can be duplicated once a reusable image is available. In cloud computing, this is closely related to horizontal scaling, where multiple instances of the same application are started to handle more requests or improve availability. The important condition is that each instance needs a separate name and a valid network mapping, otherwise the containers may conflict with existing resources on the host.



Step 9: Verify Multiple Active Ports

After starting the second container, I checked the Codespaces Ports area and confirmed that both 8080 and 8081 were active. Opening both forwarded addresses showed identical web pages, because both containers were created from the same Docker image. This verifies that one image can be used to produce multiple consistent service instances. The screenshot also demonstrates how cloud development platforms help manage network access visually. Each exposed port becomes an entry point to a running service. This step highlights both scalability and port management: scaling is easy at the container level, but users still need to track which host ports correspond to which container instances.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8080:80 --name second-container cloud-lab-image:v1
docker: Error response from daemon: Conflict. The container name "/second-container" is already in use by container "4ad7f5bcec1f7cdb44cc7236f03b45b6148062f7b0cdec1d12ced46b94276989". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
```

Step 9: Observe Port Conflict Error

I intentionally tried to start another container using host port 8080 again. Docker returned an error saying that the port was already allocated. This result is expected because the Codespaces host can bind only one service to the same host port at a time. Even if the containers are based on the same image, their exposed host ports must be managed carefully. This step provides a practical example of a common cloud networking problem. Cloud engineers can scale applications by running multiple instances, but they also need load balancers, routing rules, or different port mappings to distribute traffic correctly and avoid address conflicts.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker stop my-web-container second-container
my-web-container
second-container
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker rm my-web-container second-container
my-web-container
second-container
```

Step 10: Clean Up Containers

After finishing the container experiments, I stopped and removed the running container instances. Stopping a container ends the running process, while removing it deletes the container object from the Docker environment. This cleanup step is important because unused containers can continue occupying names, ports, memory, and other resources. In a real cloud platform, keeping unnecessary resources running may also create extra cost under a pay-as-you-go billing model. Running `docker ps` after cleanup verifies that no active containers remain. This step reflects a professional cloud operation habit: resources should be released when they are no longer needed.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

Step 11: Save Changes with Git

Finally, I used Git commands to save the lab changes back to the remote repository. The `git add` command stages the modified files, `git commit` records them as a versioned snapshot with a message, and `git push` uploads the commit to GitHub for permanent cloud storage. This final step connects the container experiment back to the software development workflow. The Dockerfile, edited web page, and related lab files are not only used temporarily in Codespaces; they are preserved in the repository so they can be reviewed, reused, or deployed again later. This demonstrates how cloud-based development combines source control, containerization, and remote collaboration.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 594 bytes | 594.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/qzxwj/cloud-computing-lab-github
   cce89e5..cfb22e7  main -> main
```

四、Reflection

1. Problems Encountered and Proposed Solutions

(1) During the experiment, one possible problem was that the Docker environment in Codespaces might not work correctly at the beginning. When I used the `docker version` command, the expected result was to see both Client and Server version information. If only the Client part appeared, or if the terminal showed that it could not connect to the Docker daemon, it would mean that the Docker service was not running properly. To solve this problem, I would refresh the Codespaces page, restart the Codespace, and run the command again to confirm the environment status. This

step reminded me that before building or running any container, the basic Docker environment must be checked first, because all later operations depend on it.

(2) Another problem was related to the creation and use of the Dockerfile. Since the file name is case-sensitive, writing it as `dockerfile`, `DockerFile`, or `Dockerfile.txt` may cause Docker to fail when building the image. In addition, the `index.html` file must be placed in the correct project directory, otherwise the `COPY index.html /usr/share/nginx/html/index.html` instruction cannot find the source file. To solve this problem, I would carefully check the file name, confirm that there is no file extension, and make sure that `index.html` and `Dockerfile` are in the same directory before running the build command. This problem helped me understand that small mistakes in file names and paths can directly affect the container image-building process.

(3) A third problem was the port conflict that may occur when running multiple containers. For example, after one container has already used host port 8080, trying to start another container with the same host port will cause Docker to report that the port is already allocated. This means that one host port can only be mapped to one running service at the same time. To solve this problem, I would use another available host port, such as 8081, and map it to the container's internal port 80. This helped me understand that port mapping is not only a command-line operation, but also an important part of cloud service access and resource management.

2. Explain why your website didn't update automatically after editing the code, and why this is actually a "feature" in cloud computing.

(1) The website did not update automatically after editing the source `index.html` file because the running container was not directly using the latest file from the project directory. When the Docker image was built, Docker copied the current version of `index.html` into the Nginx web directory inside the image. After the container started, it served the file that had already been packaged into the image. Therefore, changing the source file in the host workspace did not change the file inside the running container. To make the new content appear on the website, I need to rebuild the Docker image and restart the container.

(2) This behavior reflects the concept of immutable infrastructure in cloud computing. A Docker image is like a fixed snapshot of an application and its runtime environment. Once the image is built and the container is started, the running service should remain stable unless a new image is created and deployed. This design prevents unexpected

changes from affecting a running service. In real cloud systems, this is very useful because developers and operators can clearly know which version of the application is being deployed. It also makes rollback easier, because an older image can be reused if the new version has problems.

(3) This feature also improves the reliability and consistency of cloud application deployment. If a website changed immediately every time a source file was edited, the running environment could become unstable and difficult to control. With Docker, every change needs to go through a clear process: modify the source code, rebuild the image, stop the old container, and run a new container. Although this adds one more step, it makes the deployment process more predictable. The same image can run in different environments and produce the same result, which is one of the main reasons why containers are widely used in cloud computing.

附录 程序命令代码

附录 1: docker_command
<pre># ===== # Docker Containerization Lab Commands # 使用 Docker 进行容器化实验命令汇总 # ===== # 1. Verify Docker Environment # 验证 Docker 环境是否正常运行 docker version # 2. Create Dockerfile # 在项目根目录中新建文件，文件名必须为 Dockerfile # Dockerfile 内容如下： FROM nginx:alpine COPY index.html /usr/share/nginx/html/index.html EXPOSE 80 # 3. Build Docker Image # 构建 Docker 镜像</pre>

```
docker build -t cloud-lab-image:v1 .
```

4. Run the First Container

运行第一个容器，将主机 8080 端口映射到容器 80 端口

```
docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1
```

5. Access the Website

在 Codespaces 中打开 8080 端口，即可访问网页

6. Modify index.html

修改 index.html 文件中的内容，例如：

Success! -> Updated!

注意：直接刷新浏览器，网页不会自动更新

7. Rebuild Image After Modification

如果想让修改后的 index.html 生效，需要重新构建镜像

```
docker build -t cloud-lab-image:v1 .
```

8. Stop and Remove the Old Container

停止并删除旧容器

```
docker stop my-web-container
```

```
docker rm my-web-container
```

9. Run the Updated Container Again

使用重新构建后的镜像再次运行容器

```
docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1
```

10. Enter the Container

进入正在运行的容器内部

```
docker exec -it my-web-container sh
```

11. View Files Inside the Container

查看容器内部 Nginx 网页目录

```
ls /usr/share/nginx/html/
```

12. View index.html Inside the Container

查看容器内部 index.html 的具体内容

```
cat /usr/share/nginx/html/index.html
```

13. Exit the Container

退出容器内部终端

```
exit
```

14. Monitor Container Resource Usage

查看容器 CPU 和内存占用情况

```
docker stats
```

15. Stop Resource Monitoring

按 Ctrl + C 停止 docker stats 监控

Ctrl + C

16. Run a Second Container Instance

使用同一个镜像运行第二个容器，但主机端口改为 8081

```
docker run -d -p 8081:80 --name second-container cloud-lab-image:v1
```

17. Trigger Port Conflict Error

故意再次使用 8080 端口运行第三个容器，会产生端口占用错误

```
docker run -d -p 8080:80 --name third-container cloud-lab-image:v1
```

18. Expected Error Message

可能出现的错误提示：

```
# Bind for 0.0.0.0:8080 failed: port is already allocated.
```

19. Stop Running Containers

停止第一个和第二个容器

```
docker stop my-web-container second-container
```

20. Remove Containers

删除第一个和第二个容器

```
docker rm my-web-container second-container
```

21. Check Running Containers

查看当前正在运行的容器

```
docker ps
```

22. Check All Containers

查看所有容器，包括已经停止的容器

```
docker ps -a
```

23. Save Changes with Git

将实验修改添加到暂存区

```
git add .
```

24. Commit Changes

提交实验修改

```
git commit -m "Finished Lab 02"
```

25. Push to GitHub

推送到 GitHub 仓库

```
git push
```